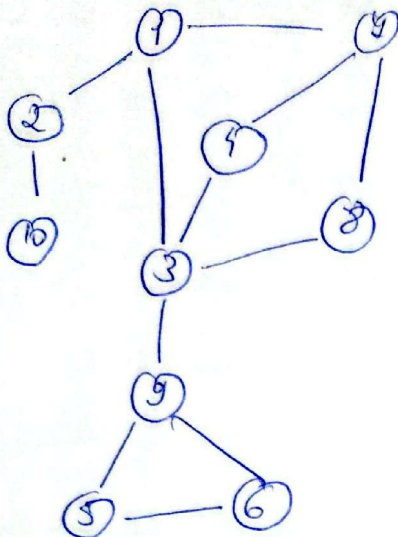




max/min = min

1) Muchie critică

$$= \{(2,10), (1,2), (3,9)\}$$

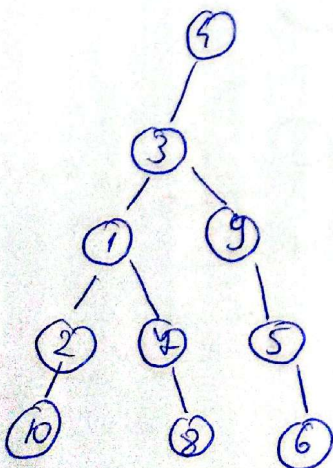
Muchie critică - dacă este eliminată graful nu mai rămâne conex.

Se află cu ajutorul algoritmului lui Tarjan (pt componente conex) modificat, în care se calculează timpurile de intrare și valorile.

Muchie critică = muchie a cărei eliminare **mărește** nr de componente conexe
și nu aparține niciunui ciclu

2) Ilustrați $df(4)$ - parcurgerea în adâncime

$df(4)$



6
5
9
2
4
10
2
1
3
4

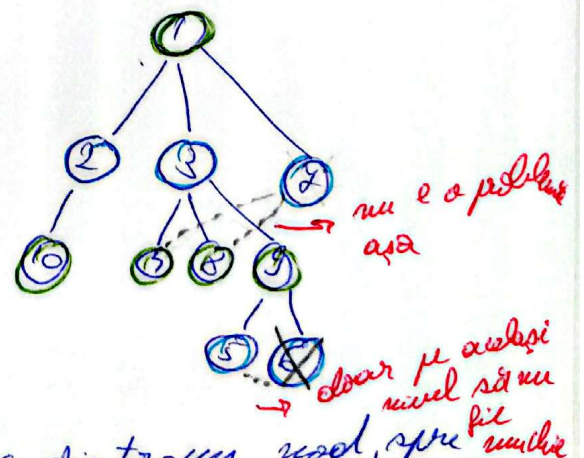
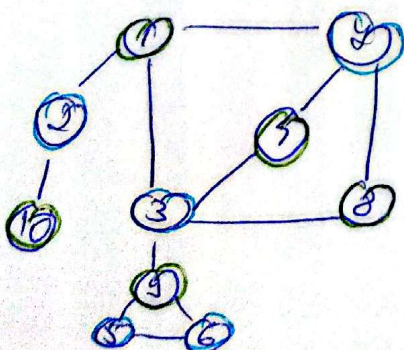
Parcurgerea în adâncime DFS pleacă de la nodul de start după care parcurge recursiv tot.

vecini nodului, în ordine lexicografică. 4 are ca vecini pe 3 și pe 7, și se duce prima dintre ele la 3. 3 devine nodul curent, este pus în stivă, și se face din nou parcurgerea pe el. Acesta 2 are ca vecini pe 1, 3 și 5. Merge în 1 care devine nod curent și se tot repetă până nu mai are vecini. Când se ajunge aici nodurile sunt scoase din stivă până când se găsește un nod care mai are vecini nevizitați, și se repetă pașii. De ex, după ce sunt vizitate nodurile $1 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 10$, 10 este eliminat din stivă (nu mai are vecini) și 2, iar la 2 este găsit 4 care este nevizitat și se repetă pașii de acolo până sunt vizitate toate nodurile.

3) subgraf lipsit de cicluri cu nr maxim de noduri

BFS (1)

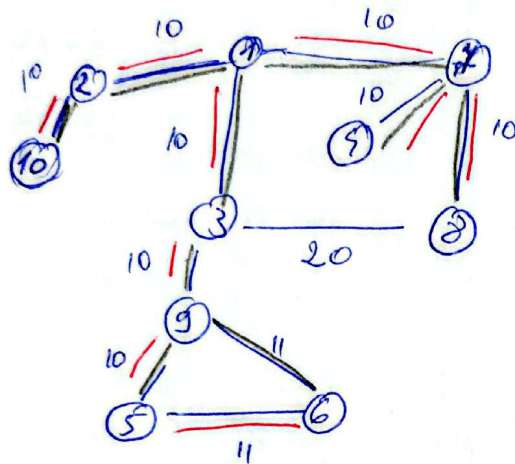
1	2	3	4	10	4	8	9	5	6
---	---	---	---	----	---	---	---	---	---



Făcem BFS dintr-un nod, spre ex 1, pentru a vedea arborii de parcurgere. Colorăm nodurile pe fiecare nivel și vedem care muchii

din graful nostru original
formarea cicluri. ~~Apa alegem și~~
~~eliminăm nodul 7 care face ciclul~~
~~cu 4, 3 și 8 și anul din nodurile~~
care sunt fiice lui 3, 5 sau 6.

4) 2 APM de cost minim de 50 $\rightarrow$ se determină cu
Kruskal sau Prim.



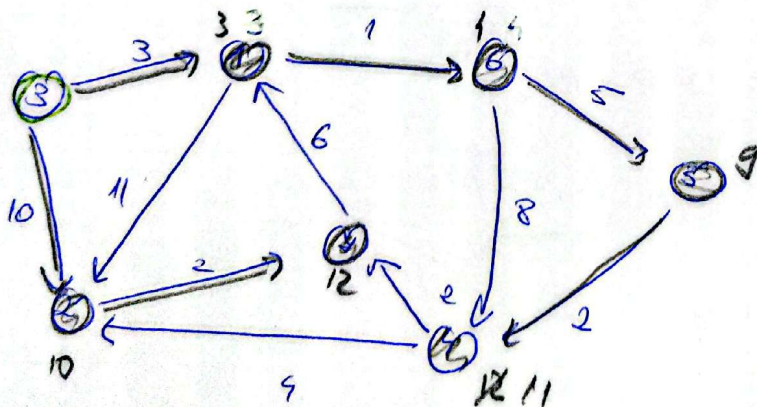
APM₁: (10, 2), (2, 1), (1, 4), (4, 9),
(9, 7), (11, 3), (3, 9), (9, 5),
(9, 6)

cost = 91

APM₂: (10, 2), (2, 1), (1, 4), (4, 9),
(4, 8), (1, 3), (3, 9), (9, 5),
(5, 6)

cost = 91

5) Dijkstra (3)

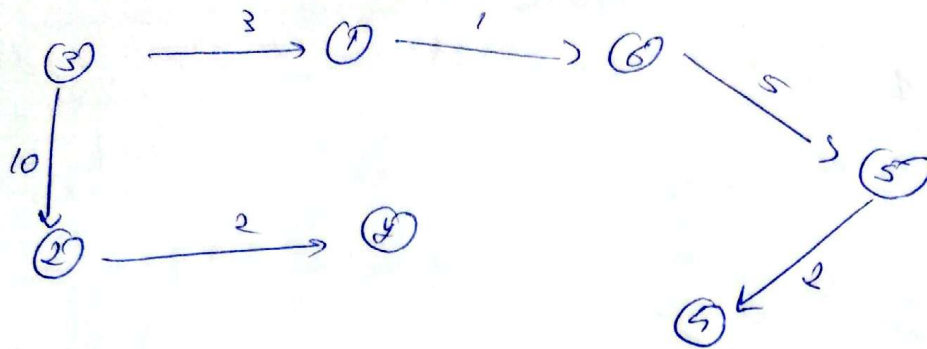


Dijkstra porneste din vârful de start 3 și
actualizează vectorul de drumuri minime găsite
până în acest moment și pe cel de față, astfel,

din 3 se poate ajunge la 1 cu costul minim
 3, iar la 2 cu costul 10. Se alege minimum din
 acesta, adică drumul până la 1. Când se
 ajunge în nodul 1 se relaxează toate
 muchiile care pleacă din 1, adică se verifică dacă
 se găsesc prin 1 drumuri mai scurte la alte
 noduri, deci vedem că în 2 putem ajunge prin $3+11=14$,
 dar nu este mai mic decât 10 pe care îl stăruie deja
 nu actualizăm, dar putem ajunge în 6 cu $3+1=4$
 care este mai mic decât ∞ și actualizăm. Se repetă
 pașii până se găsesc toate drumurile minime.

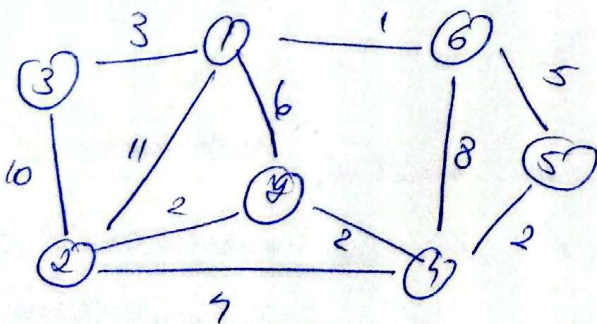
	1	2	3	4	5	6	7
distanță	∞	∞	∞	∞	∞	∞	∞
	<u>3/3</u>	10/3	0/0	∞	∞	∞	∞
n=1	—	10/3	0/0	∞	∞	<u>4/1</u>	∞
n=2	—	10/3	0/0	12/6	<u>9/6</u>	—	∞
n=3	—	<u>10/3</u>	0/0	11/5	—	—	∞
n=4	—	—	0/0	<u>11/5</u>	—	—	12/2
n=5	—	—	0/0	—	—	—	<u>12/2</u>
n=6	—	—	0/0	—	—	—	—

Graficul de drumuri minime



6) Alg. Kruskal - se face sortarea muchilor grafului.

- se iau muchiile în ordinea sortată crescător
- se verifică dacă muchia aleasă nu duce la formarea unui ciclu în APM sau (altfel partial de cost minim)
- dacă nu este adăugată în APM, altfel este ignorată

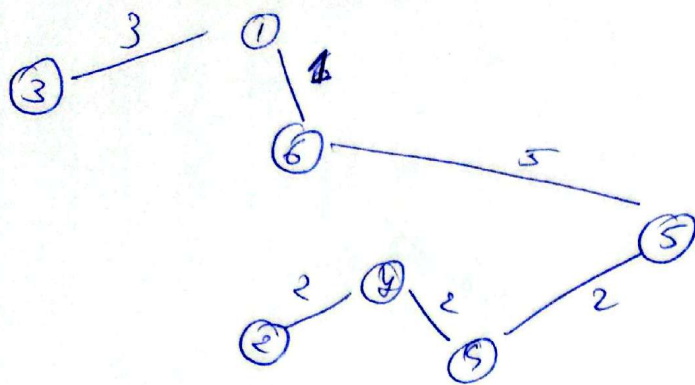


(3,1), (3,2), (1,6), (6,5), (5,4)

(6,4), (4,3), (4,2), (2,3), (1,4), (1,2)

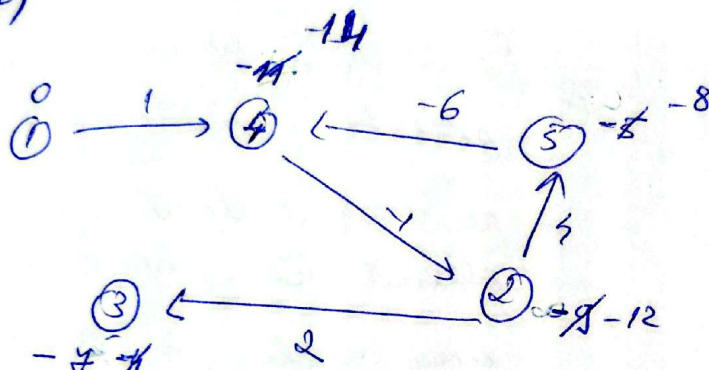
(1,6), (4,4), (4,2), (5,4), (3,1), (2,4), (6,5), (1,4), (6,4), (3,2), (1,2)

Formăm APCM.



$\{(1,6), (4,2), (2,4), (4,5),$
 $(1,3), (5,6)\}$

7)



$E = \{(1,4), (2,3), (4,2), (2,5), (5,4)\}$

	1	2	3	4	5
d	0	-8	-7	-14	-8
		-12	-7	-14	-8

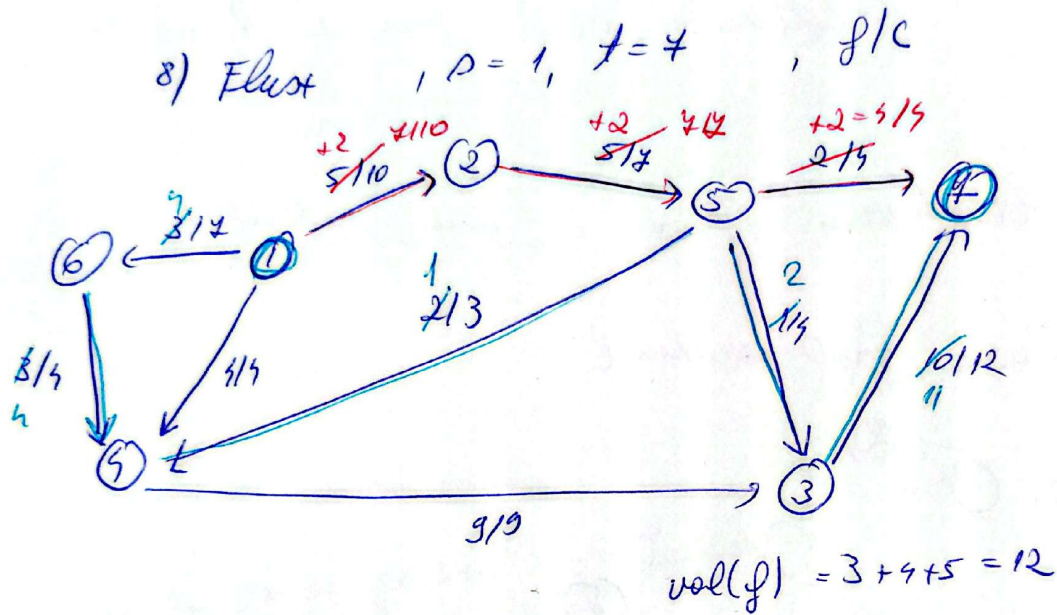
- în pseudocod mai adăugăm încă o relaxare a muchiilor și dacă găsim el putem un drum care se modifică însemnând că suntem într-un ciclu:

pentru fiecare $uv \in E$ exista
dacă $d[u] + w(u,v) < d[v]$

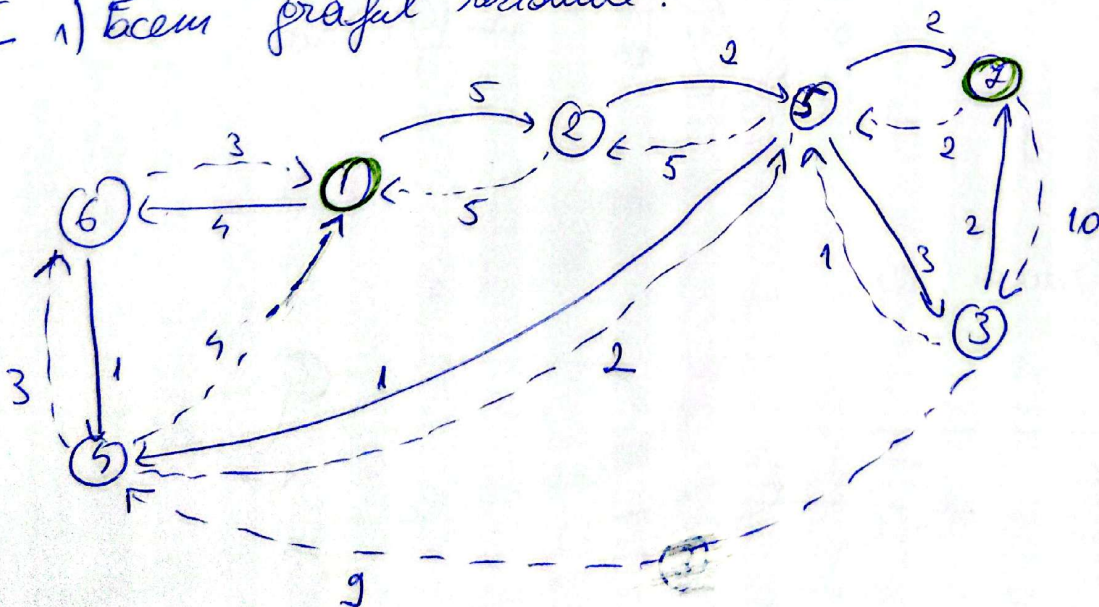
atunci

se înregistrează 'ciclu negativ'

- vedem că în exemplul nostru dacă mai facem o relaxare drumurile se tot modifică, cîntal pînă în 2 este -9 și în 3 este -4, dar dacă relaxăm muchia (2,3), drumul pînă în 3 se modifică $-9+2=-7 < -4 \rightarrow$ actualizăm $d[3]=-7$.



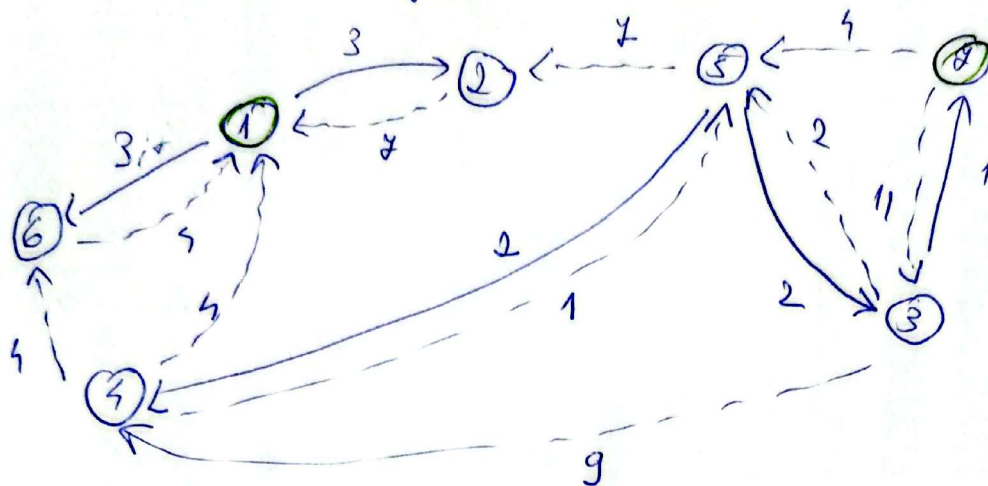
I 1) Facem graful rezidual.



2) Facem BFS pe graful rezidual din $s=1$.

3) Actualizăm fluxul.

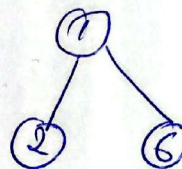
III 1) Refacem graficul residual.



2) Travers BFS

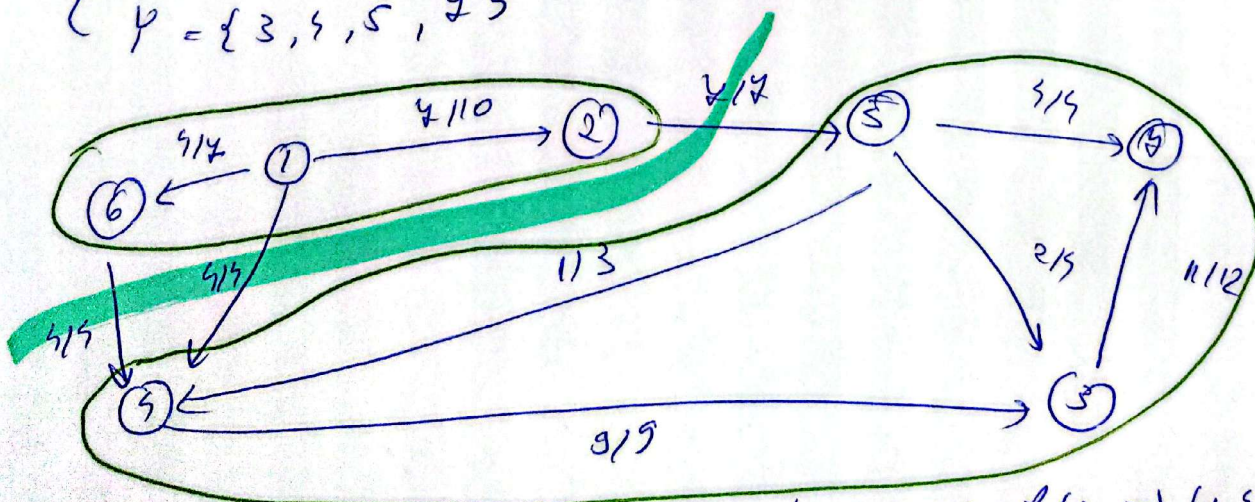
BFS

126



→ nu mai facem drum

$$\text{val}(f)_{\max} = 9 + 9 + 7 = 15$$

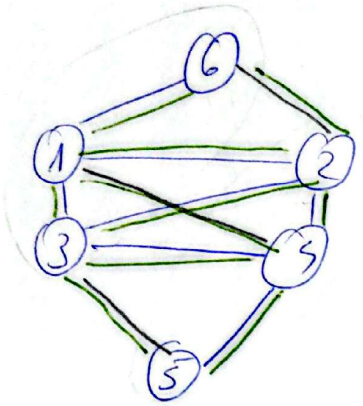
$$\begin{cases} X = \{1, 2, 6\} \\ Y = \{3, 4, 5, 7\} \end{cases} \quad \text{bipartite}$$


taichura min = $7 + 9 + 9 = 15$

arc direct $(2, 5), (1, 9), (6, 4)$ ✓
 arc reverse $\emptyset$

9) a) 6 graf neorientat care are exact h noduri de grad impar.

Arătați că există exact $\lfloor \frac{h}{2} \rfloor$ componente simple



$h=6$

- imparații ale h și $\lfloor \frac{h}{2} \rfloor$ este
- adăugăm câte 2 muchii între u și v pentru u, v prochi
- $\Rightarrow$ toate gradele devin pare
- grafurile devin euleriene
- $\Rightarrow$ există un ciclu eulerian
- care folosește toate muchiile
- scot muchiile artificiale:
- ciclu se ține în $\lfloor \frac{h}{2} \rfloor$ componente

~~5-3-1-4-3-2-4-6-2-4-5~~

~~5-3-1-4-3-2-4-6-2-4-5~~

3-2-1

4-5

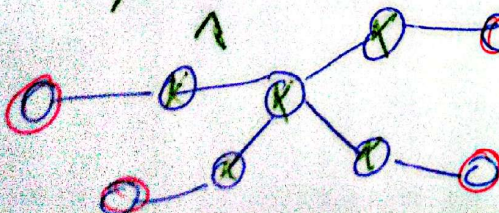
6

b) Fie $n \geq 5$. Care e nr max noduri critice ale unui graf neorientat cu $h=4$ noduri de grad impar?

! Obs - într-un arbore, orice nod care nu e frunză e pt critic.

- nodurile de grad impar, în special frunzele, nu sunt puncte critice

pt critic



$\rightarrow$ 4 noduri impare

nr max pt critic = $n-4$

10) Distanța de editare

inserare = c_1 $\Rightarrow$ ștergere = c_1 (inserare inversă)
modificare = c_2

1) initializare

$$c[i][0] = i \cdot c_1$$

$$c[0][j] = j \cdot c_1$$

2) recurență : $\exists a[i] = b[j]$

$$c[i][j] = c[i-1][j-1]$$

$$\nexists a[i] \neq b[j]$$

$$c[i][j] = \min \begin{cases} \text{ștergere} = c[i-1][j] + c_1 \\ \text{inserare} = c[i][j-1] + c_1 \\ \text{modificare} = c[i-1][j-1] + c_2 \end{cases}$$

11.2) Algoritmul de 6 - colorare $\rightarrow$ graf planar conex

I $n \leq 6$ - se colorează fiecare nod diferit

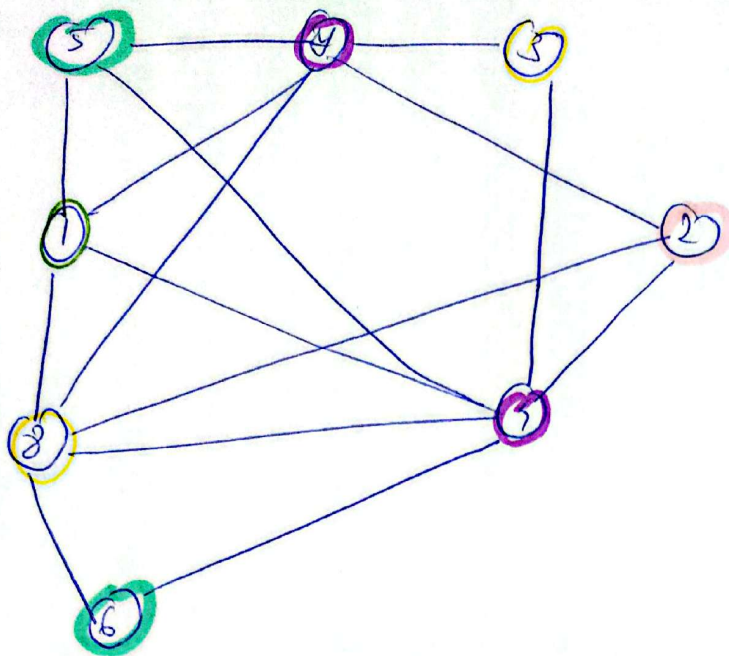
II $n > 6$ - se extrage nodul x cu $\text{grad} \leq 5$

- se elimină din graf

- se colorează subgraful rămas

- se adaugă nodul în graf și se

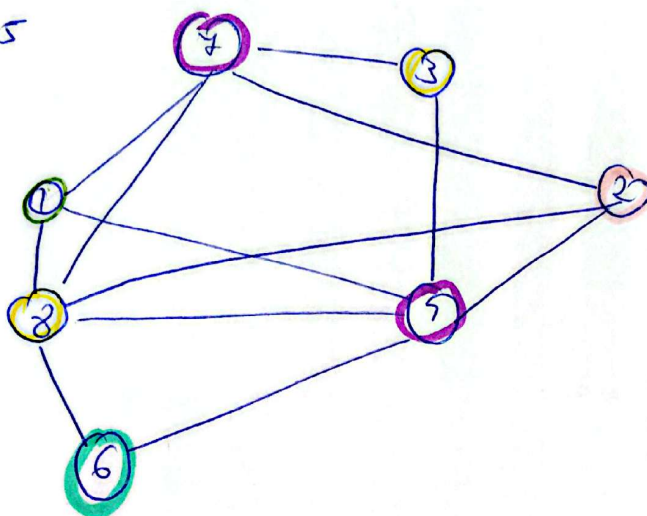
colorează dif față de vecini.



Adăugăm 5 și
 îl colorăm diferit
 vecinilor {1, 4}

$\Rightarrow$ culoare 2, 3 sau 6

el. 5

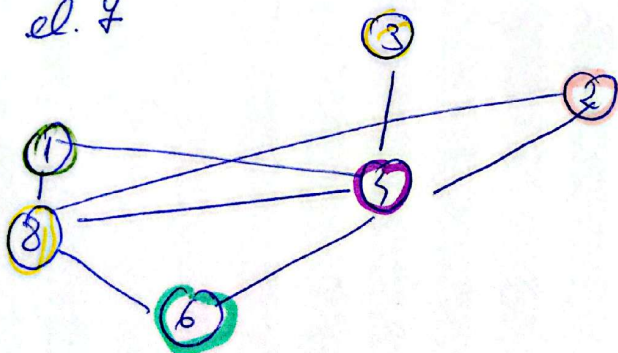


Adăugăm nodul
 7 și îl colorăm
 diferit de vecinii

{1, 3, 2, 5}

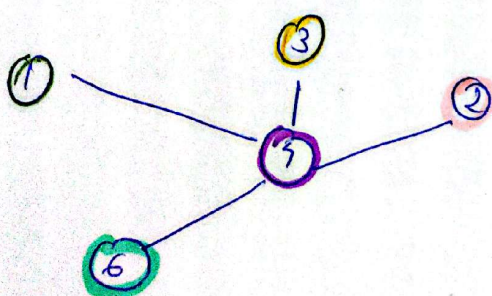
$\Rightarrow$ culoare 4

el. 4



Adăugăm nodul 4
 și îl colorăm diferit
 de vecinii {1, 2, 5, 6} $\uparrow\uparrow$
 $\rightarrow$ culoare 3

el. 3



Acum poate
 fi fiecare nod colorat
 diferit.

12/